Assignment #1

전기전자공학부 202110939 신주엽

Assignment_CC

```c
#include <stdio.h>
#include <stdlib.h>

#define MAX 225

typedef struct {
    int x, y;
    int dir;
} Data;

Data stack[MAX];
int top = -1;

void init_stack(void)
{
    top = -1;
}

void push(Data new)
{
    if (top >= MAX - 1)
    {
        printf("Stack overflow !!!₩n");
    }
    stack[++top] = new;
}

Data pop()
{
    if (top < 0)
    {
        printf("Stack underflow !!₩n");
        return (Data) { -1, -1, -1 };
    }
    return stack[top--];
}
```

이 코드는 Array-based Stack을 기반으로 한 연결 요소 탐색(Connected Component Analysis)을 위한 기본적인 스택 자료구조 구현부로, 사용자 정의 구조체 Data를 통해 (x, y) 좌표와 방향 정보를 함께 저장할 수 있도록 하였다. Stack의 최대 크기는 15*15인 225로 설정하였다. stack 배열과 top 포인터를 사용하여 스택의 상태를 관리하며, init_stack() 함수는 스택 초기화를 담당한다. push() 함수는 새로운 데이터를 스택에 추가하며, 스택이 가득 찼을 경우 오버플로 경고를 출력한다. pop() 함수는 스택에서 데이터를 꺼내며, 스택이 비어 있을 경우 언더플로 경고와 함께 기본 값을 반환한다.

```
int dx[8] = { -1, -1,  0, 1, 1,  1,  0, -1 };  // x축(세로) 방향: 위에서 아래로
int dy[8] = { 0,  1,  1, 1, 0, -1, -1, -1 };  // y축(가로) 방향: 왼쪽에서 오른쪽으로

void find_1(int location[15][15])
{
    int label = 2;

    for (int x = 0; x < 15; x++) {
        for (int y = 0; y < 15; y++) {
            int found_1 = 0;
            for (int d = 0; d < 8; d++) {
                int nx = x + dx[d];
                int ny = y + dy[d];

                if (nx >= 0 && nx < 15 && ny >= 0 && ny < 15 && location[nx][ny] == 1) {
                    found_1 = 1;
                    break;
                }
            }
            if (found_1) {
                int cx = x;
                int cy = y;

                for (int d = 0; d < 8; d++) {
                    int nx = cx + dx[d];
                    int ny = cy + dy[d];

                    if (nx >= 0 && nx < 15 && ny >= 0 && ny < 15 && location[nx][ny] == 1) {
                        push((Data) { cx, cy, d });
                        location[nx][ny] = label;
                        cx = nx;
                        cy = ny;
                        break;
                    }
                }
                while (top >= 0) {
                    int found = 0;

                    for (int d = 0; d < 8; d++) {
                        int nx = cx + dx[d];
                        int ny = cy + dy[d];

                        if (nx >= 0 && nx < 15 && ny >= 0 && ny < 15 && location[nx][ny] == 1) {
                            push((Data) { cx, cy, d });
                            location[nx][ny] = label;
                            cx = nx;
                            cy = ny;
                            found = 1;
                            break;
                        }
                    }
                    if (!found) {
                        Data before = pop();
                        cx = before.x;
                        cy = before.y;
                    }
                }
                label++;
            }
        }
    }
}
```

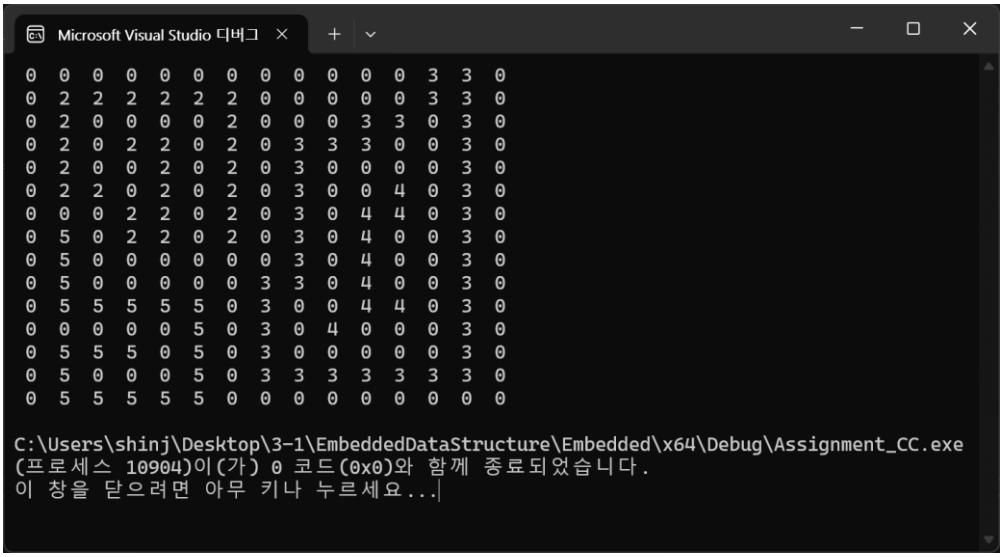
find_1() 함수는 2차원 배열에서 값이 1인 모든 연결 요소(connected components)를 탐색하고, 각 요소를 2 이상의 라벨로 바꾸는 함수이다. 전체 배열을 (0,0)부터 (14,14)까지 순차적으로 탐색하며, 현재 위치 주변 8방향에 1인 값이 존재할 경우 해당 위치에서 탐색을 시작한다. 스택에 현재 위치 정보를 push하고 연결된 모든 1을 label로 갱신한다. 더 이상 연결된 1이 없을 경우 pop()을 이용해 직전 위치로 돌아가 다시 탐색을 이어간다. top>=0 조건을 만족하지 못하면 하나의 컴포넌트 탐색이 끝났다는 의미이므로 label++을 통해 다음 컴포넌트를 위한 라벨을 준비하고 다음 컴포넌트를 찾아서 반복한다.

```
int input_map[15][15] = {
        {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0},
        {0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1, 1, 0},
        {0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 0},
        {0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 0, 0, 1, 0},
        {0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0},
        {0, 1, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 0, 1, 0},
        {0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0},
        {0, 1, 0, 1, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 0},
        {0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 0},
        {0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 0, 1, 0},
        {0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 0, 1, 0},
        {0, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0},
        {0, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 0},
        {0, 1, 0, 0, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0},
        {0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0}
    };

void main(void) {

    find_1(input_map);

    for (int i = 0; i < 15; i++) {
        for (int j = 0; j < 15; j++) {
            printf("%2d ", input_map[i][j]);
        }
        printf("₩n");
    }
}
```



함수를 호출하고 출력하면 위와 같이 [1][1]에 위치한 요소부터 라벨링을 진행하며 2, 3, 4, 5까지 connected component를 라벨링을 한 것을 확인 할 수 있다. 이번 과제는 Array-based Stack을 사용하였지만 Linked list-based Stack으로도 해보는 것도 학습에 도움이 될 것 같다.